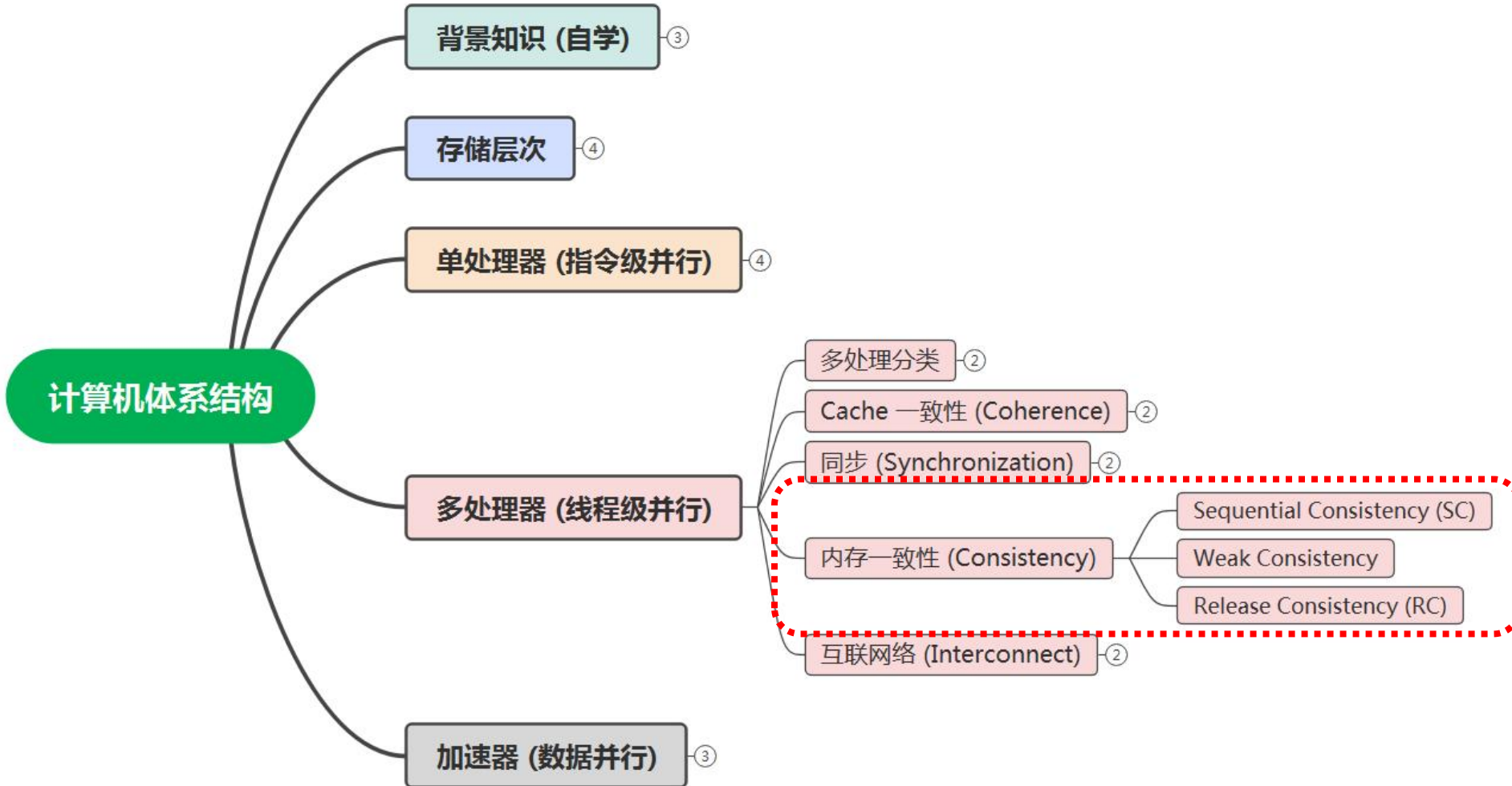


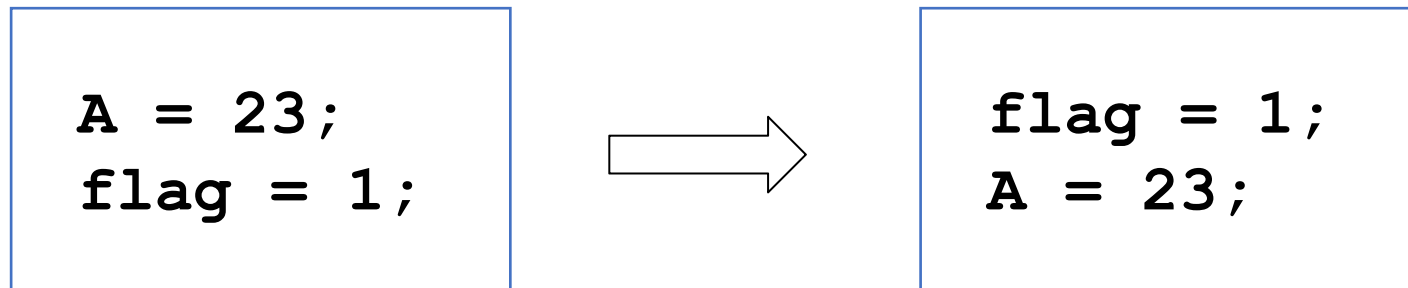
计算机体系结构

L06-3 多处理器内存一致性



什么是内存一致性

- 单处理器可以在无相关性的前提下乱序执行指令



这是安全的, 因为没有数据相关

注意: 改变指令顺序的约束条件 → 保证没有相关性

- 数据相关性必须解决
- 控制相关性必须解决

- 改变指令顺序可能由编译器或者处理完成

什么是内存一致性

□ 乱序执行在多处理器场景可能会引起问题

```
A = Flag = 0
```

P1

```
A = 23;
```

```
Flag = 1;
```

P2

```
while (Flag != 1) {};
```

```
... = A;
```

如果P1的两条指令顺序执行，P2读到的A一定是23

什么是内存一致性

□ 乱序执行在多处理器场景可能会引起问题

```
A = Flag = 0
```

<u>P1</u>		<u>P2</u>
A = 23;		while (Flag != 1) {};
Flag = 1;		... = A;

如果P1的两条指令乱序执行(Flag = 1先于A = 23执行) , P2读到的A可能是0 !

什么是内存一致性

- 单处理器可以在无相关性的前提下乱序执行指令
- 然而, 这种乱序执行在多处理器场景可能会引起问题
- **问题:** 为了确保并行程序按照程序员的预期执行, 我们必须对指令重排序施加哪些约束?
- 比较重要的一个约束就是**内存一致性**
 - *Memory Consistency Model*

内存一致性模型

□ 硬件/编译器和程序员之间的一个约定

- 硬件和编译器在指令重排中不能违反个约定
- 程序员也需要遵循约定的规则

□ Example: 如果程序员想从P2获得23

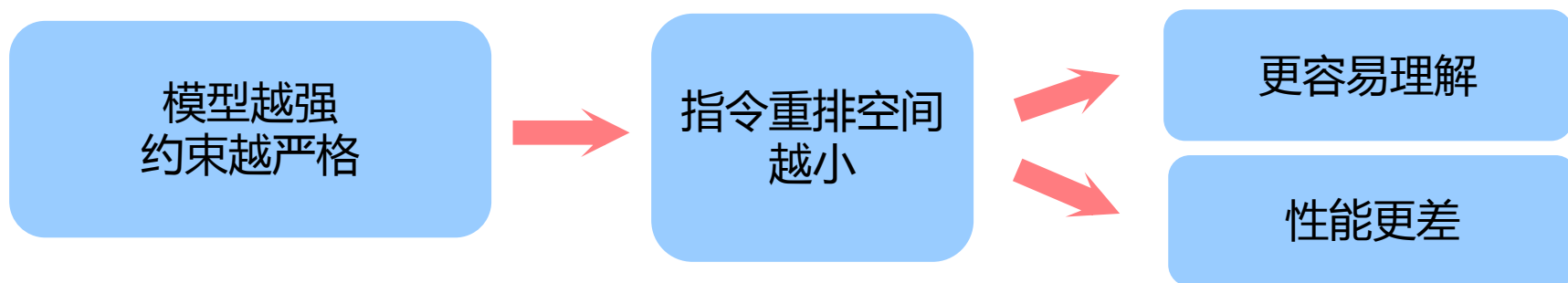
P1
`A = 23;`
`Flag = 1;`

P2
`while (Flag != 1) {};`
`... = A; // get 23`

为了满足这个执行顺序, 硬件或编译器不能进行某些优化,
e.g. load bypassing → **会影响执行效率**

内存一致性模型

- 设计好的一致性模型需要在**可编程性和性能之间做好权衡**
- 保持“预期的”执行顺序通常会简化程序员的工作
 - Ease of debugging, state recovery, exception handling
- 保持“预期的”执行顺序往往会使硬件设计师在优化时面临困难



顺序一致性 (Sequential Consistency)

□ 一个多处理器系统被称为**sequentially consistent (SC)**,
如果满足下面两个条件：

- 所有处理器的操作按照某种顺序执行
- 来自同一个处理器的操作**按照指令顺序执行**

□ **这是一种内存顺序模型, 或内存模型**

- 所有处理器看到的内存指令执行顺序相同, i.e., 所有内存操作按照某种顺序执行 (称为**global total order**)
- 在这个执行顺序中, 每个处理器的操作按照指令顺序执行

顺序一致性示例

P1

a1: x = 1;

a2: y = 1;

P2

b1: R_y = y;

b2: R_x = x;

b1: R_y = y;

b2: R_x = x;

a1: x = 1;

a2: y = 1;

b1: R_y = y;

a1: x = 1;

b2: R_x = x;

a2: y = 1;

b1: R_y = y;

b2: R_x = x;

a2: y = 1;

a1: x = 1;

b1: R_y = y;

b2: R_x = x;

a1: x = 1;

a2: y = 1;

≡

(R_x=0, R_y=0)

a2: y = 1;

b1: R_y = y;

b2: R_x = x;

a1: x = 1;

顺序一致性

□ 简单、直观

- 与程序员的直觉一致
- 容易解释程序行为

□ 在同一次执行中, 所有处理器看到相同的内存指令执行顺序

- 无正确性问题

□ 如果多次执行, 每次执行可以观察到不同的执行顺序 (每次都是sequentially consistent)

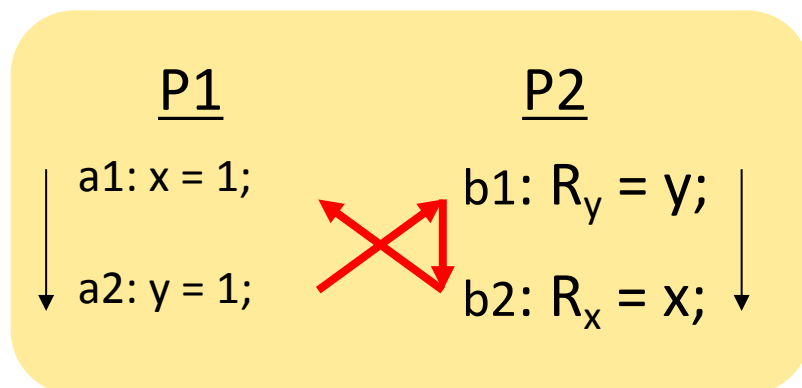
- 调试仍然很困难 (每次执行指令顺序不一样)

Coherence vs. Consistency

- Coherence 仅关注一个内存地址的值
- Consistency 关注所有内存地址的访问顺序
- A memory system is *coherent* if
 - 能够将访问该地址的操作序列化
 - 每个处理器的操作都按照指令顺序执行
 - Read返回最新的值
 - 不能保证最新的数据何时能被看到
 - 不能保证数据更新以什么顺序被看到
- A memory system is *consistent* if
 - 遵守 Memory Consistency Model
 - 内存访问顺序按照某种定义的顺序执行

Coherence vs. Consistency: Example

Parallel program



Execution sequence

a2: y = 1;
b1: R_y = y;
b2: R_x = x;
a1: x = 1;

- 该执行顺序在SC模型中可能出现么?
- 该执行顺序在coherent的多处理系统中可能出现么?
 - 什么情况会出现?
- 结论 : sequential consistency比coherence更严格

顺序一致性的问题

□ 硬件上很难高效实现

- 简单实现:
 - 无并发内存访问
 - 每个节点严格按照顺序访问内存
 - 本质上排除了乱序执行处理器

□ 不必要的限制

- 并行程序很多操作可以安全地重排序，但因为SC限制无法实现

□ 如何解决？

- 让程序员多做一些工作, e.g. 提供更明确的提示, 让硬件/编译器有更多优化空间

→ **削弱或放宽内存一致性模型**

更弱的内存一致性模型

- 程序员需要承担更多的责任,明确指示机器哪些代码序列不能被重排序,哪些代码序列可以被重排序
- 多处理器系统必须为程序员提供某种方式,例如指令或库,以实现这种指示
 - 程序员负责提供合适的指示,确保程序正确执行
- 程序员能够做的最简单的事是什么?

弱一点的模型: Weak Consistency

□ 程序员在程序中插入 *fence* 指令:

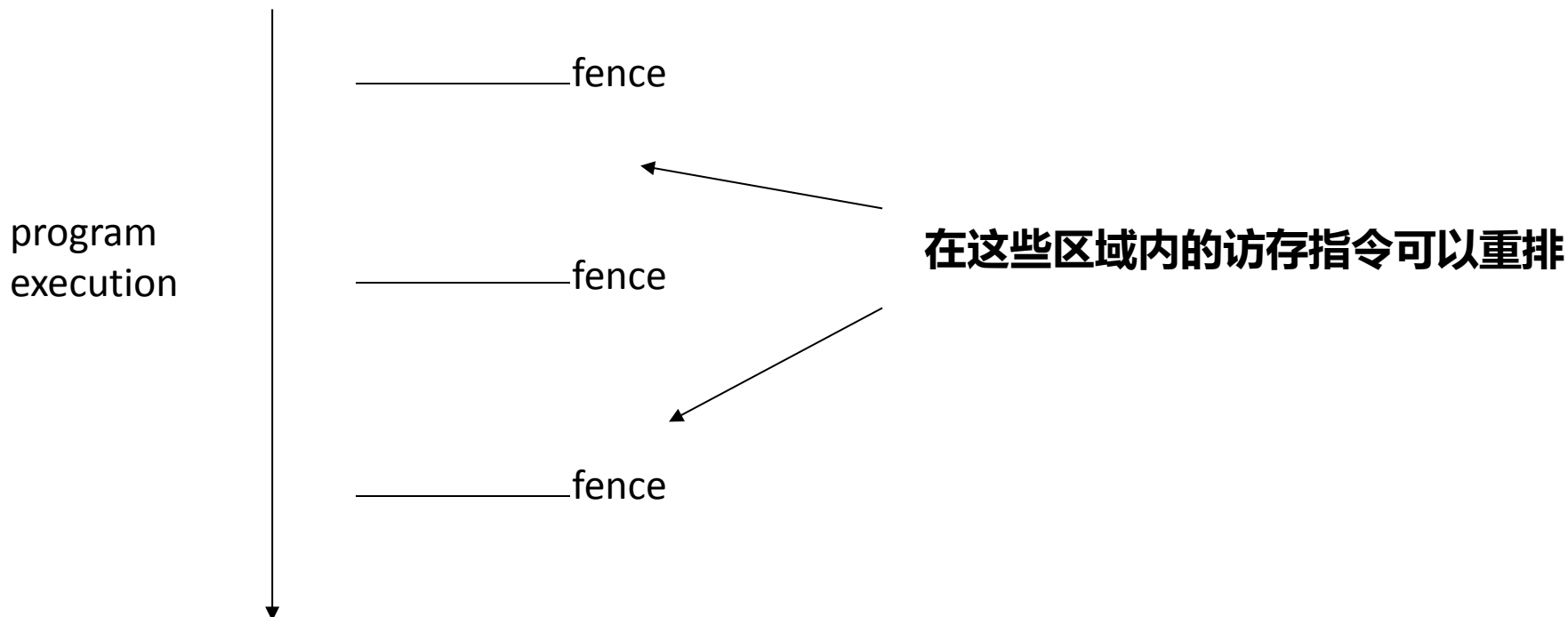
- fence之前的指令一定比fence之后的指令早完成
- 所有在fence之前的数据操作一定比fence早完成
- 所有在fence之后的数据操作一定比fence晚完成
- 所有的fence按照指令顺序完成

□ 同步原语, 例如barrier,可以用作fence

- Fence在特定的点上传播读写操作,类似刷新内存操作

P1
p = new A(...)
———— FENCE
flag = true;

Weak Ordering



□ fence硬件实现:

- 在发送fence后的下一个写操作之前，等待所有未完成的写操作完成
- 跟踪未完成写操作的逻辑可以是一个简单的计数器，该计数器在请求时递增，在响应时递减

Example of Fence

□ Initially $A = \text{Flag} = 0$

P1

$A = 23;$

fence;

$\text{Flag} = 1;$

P2

$\text{while}(\text{Flag} \neq 1) \{ \};$

$\dots = A;$

□ Execution:

- P1 写数据到 A
- Fence 等待, 直到写操作完成
- P1 写数据到 Flag
- 如果P2 看到 $\text{Flag} == 1$, 表明A的值肯定已经是23

Weak Consistency

□ Advantage

- 不用保证严格的内存操作顺序
 - 让硬件有更多的性能优化空间

□ Disadvantage

- 程序员负担加重 (need to get the “fences” correct)

更弱的模型: Release Consistency

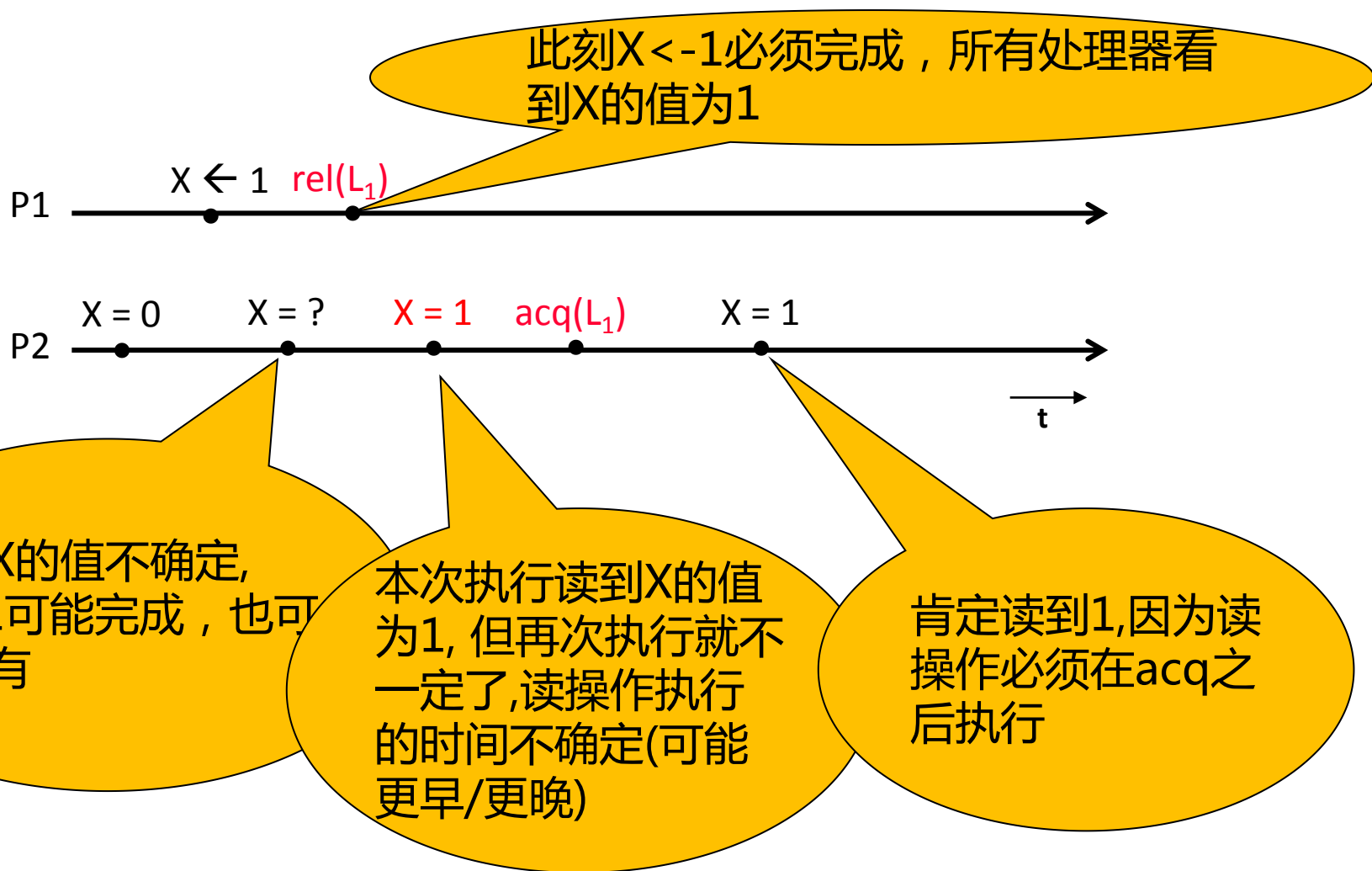
□ **将fence分成两个:** 一个保证在此之前完成, 一个保证在此之后完成

- *Acquire*: 在acquire之后的访存指令必须在acquire完成之后完成
- *Release*: 在release之前的访存指令必须在release完成之前完成

□ **However,**

- Acquire 之前的指令完成时间无限制
- Release之后的指令完成时间无限制

Release Consistency 深入理解

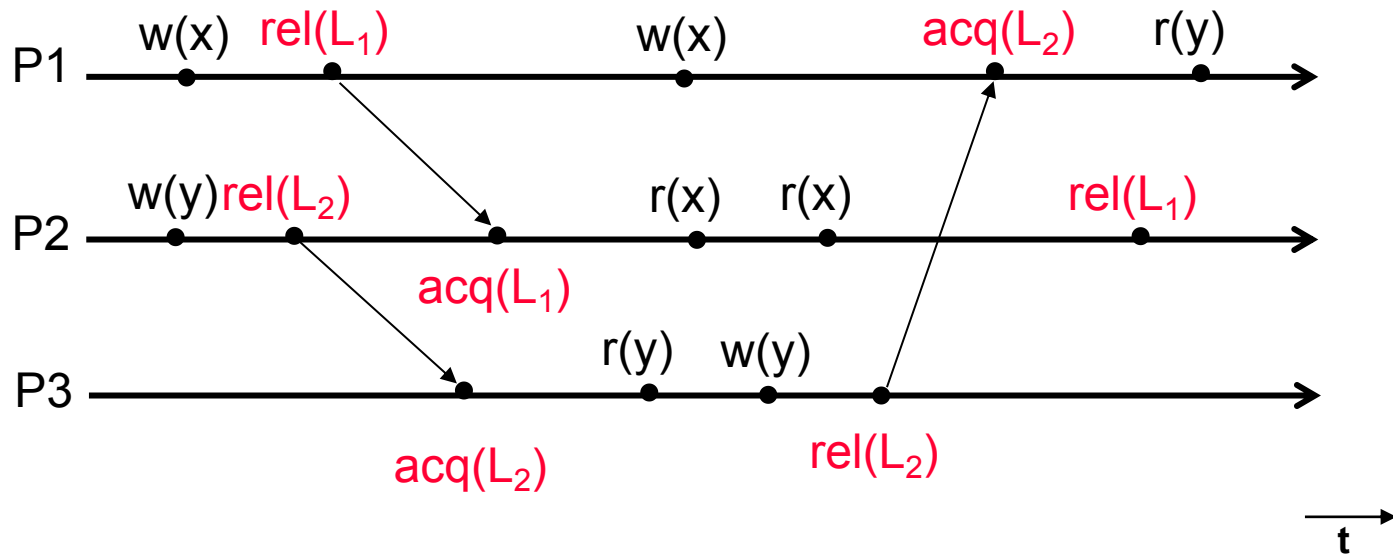


Acquire and Release

□ Acquire和Release不仅用于指令执行的同步, 也用于处理器之间内存操作的同步

- Release 起到内存同步的作用, 将本地的更新刷新到其他处理器
- Release紧跟着Acquire, 保证release之前的写操作的结果都能被acquire之后的读操作看到

Happened-Before Relation



总结

- Consistency model 是多处理特有的
- 程序员通常需要给出显式的同步提示
- Sequential consistency: 按照全局指令顺序完成访存操作
- Relaxed consistency models: 都依赖于某种类型的 fence 操作, 这种操作划定了允许指令重排的区域